Egg segmentation and tracking

Task	Nosh Robotics CV Intern Task
Author	K Pradeep Kumar
Repository	https://github.com/KpradeepKumar25/Egg-segmentation-and-tracking

Summary. (1) I made reference masks for all 160 images with OWLv2 + SAM ViT-H and reviewed them by hand. (2) I built a classical segmenter (no deep learning at inference) on a leak-free development split of 128 images; my experiment log covers six trials including the failures. (3) On the 32 held-out images it reaches F1@0.5 = 0.706 (95% CI 0.55-0.85) against the Task 1 masks. (4) I adapted the same ideas to the video, where two eggs are tracked with stable IDs through occlusion and hand movements. **Video output:** https://drive.google.com/file/d/10tqBCNuAKUDZLcVYgcEYjmvncy57eFi1/view?usp=sharing

Task	What was delivered	Headline result
1	SAM-based masks for 160 images (182 eggs), 29 images edited by hand	120 images accepted automatically; 2 images contain no egg
2	Leak-free 4:1 split, classical segmenter, experiment log T0-T5	Dev (5-fold out-of-fold) F1@0.5 0.717
3	One-shot evaluation, CIs, failure taxonomy, qualitative figures	Eval F1@0.5 0.706, fg IoU 0.674, count MAE 0.062
4	Video segmentation + tracker, public output video	2 eggs, IDs consistent after fixing a v1 swap; coverage 81% / 92%

1. Dataset and leak-free split

Data. Roboflow project *egg-vzhtr*, **version 2**, folder export: 160 images, all 640 x 640 (158 labelled 'egg', 2 'Unlabeled'). The task PDF links dataset version 1; version 2 is what was downloaded and used throughout, and the exact download call is in `src/eggcv/data.py`. I ignored Roboflow's own train/valid/test split (112/32/16). No exact byte-duplicate files were found.

Leakage control. Images of the same scene would inflate the evaluation, so I grouped images before splitting: two images are in one group if their perceptual hashes (pHash, 64-bit) differ by at most 8 bits, or if they share a Roboflow parent filename (the part before `.rf.`), merged with union-find. This gave 157 groups (3 contain more than one image, largest 2), so this export contains little near-duplication. I then split the groups with `GroupShuffleSplit(test_size=0.2, random_state=43)` into **128 dev / 32 eval images** (exactly 4:1), and the code asserts that no group appears on both sides. Everything in Task 2 (feature design, model fitting, threshold choice, every trial in the log) used the dev split only; the eval split was touched once, in Task 3, after the final model was frozen with a SHA-256 hash of the model file.

2. Task 1: SAM / pretrained-model segmentation

Method. SAM alone has no notion of 'egg'. My first attempt (SAM automatic masks, filtered by ellipse fit, solidity and skin colour) was not reliable enough to serve as a reference, so I discarded it. The final method prompts SAM with detections: OWLv2 (`owlv2-base-patch16-ensemble`) is queried with three phrases ('a photo of an egg', 'a white egg', 'a partially visible egg cut off at the edge of the image'); boxes are kept if their score is at least max(0.10, 0.4 x best score in the image) and merged with NMS (IoU 0.5); each box prompts SAM ViT-H (single mask); holes are filled and only the largest component kept; masks that fill less than 30% of their box are rejected; a final instance NMS removes duplicates. If nothing passes, a looser fallback (0.03 / 0.5) is tried.

Human review (disclosure). I checked all 160 results visually: every fallback image and every image with two or more eggs, a random sample of 12 single-egg images, and a full contact sheet of all single-egg images. I then made these edits, all recorded in `task1_sam.py`: for 14 images where nothing was found, picking the correct SAM candidate (12) or confirming that the image has no egg (2); deleting wrong eggs or adding a missed egg by one click (8 images); accepting extra eggs proposed by a low-threshold OWLv2 pass or by a click (7 images); and one image where leaves cut the egg, where all SAM pieces were kept. Final status counts: strict 120, fallback 11, manual pick 12, manual fix 8, manual add 7, no egg 2. Total **182 eggs** (dev 148, eval 34); 17 images have two or more eggs.

Caveat. These masks are SAM output corrected by me, not independent manual annotation. Task 3 therefore measures agreement with this reference, not absolute accuracy.

3. Task 2: classical segmenter and experiment log

Final method (trial T5). Propose-and-rank, using only classical image operations plus a small Random Forest: (a) compute five channels (L, a, b, saturation, and 'whiteness' = L - 1.5 x chroma), threshold each with Otsu in *both polarities*, clean with open+close (9x9 ellipse), fill holes, drop regions under 2% of the image: (b) Describe each candidate with 25 shape, edge, contrast and colour features (area, solidity, ellipse fit, border contact, edge strength, contrast, texture, colour spread, centre distance, mean L, chroma, circularity, how many other candidates agree with it, which channel produced it). (c) A Random Forest (300 trees) predicts each candidate's IoU with the reference egg. (d) The top six candidates are refined with GrabCut (4 iterations at 320 px), added to the pool, and a second Random Forest re-ranks raw plus refined candidates. (e) The best candidate is always kept; others are kept if the predicted IoU is at least 0.5 and they do not overlap a kept candidate (NMS IoU 0.3). Dev scores are out-of-fold (5-fold GroupKFold on the pHash groups) so no image is scored by a model that saw it.

Experiment log. Each row gives what changed, why, the parameters, the dev results and the decision. Failures are kept on purpose; the log also lives in `results/task2/experiment_log.md`.

Trial	What changed / why	Parameters	P	R	F1	mIoU	fgIoU	MAE	Decision
T0	Baseline: grey, global Otsu. *simplest classical pipeline; establishes a floor*	Gaussian 5x5; Otsu; open+close 5x5 ellipse; fill holes; min area 500 px	0.251	0.655	0.363	0.752	0.568	1.98	DISCARD: Assumed the egg is the bright region. Dataset has brown/dark eggs on bright walls, trays and cloth, so it picked the background.
T1	Colour cue: LAB whiteness = L - 1.5*chroma. *eggs look white/low-saturation; wanted to ignore coloured background*	as T0, whiteness map instead of grey	0.294	0.649	0.405	0.767	0.584	1.52	DISCARD: Whiteness cue is better than raw grey but still the same polarity assumption; same failure.
T2	Polarity-agnostic: 5 channels x 2 polarities, rank candidates by hand-set score. *T0/T1 showed eggs are often darker than background*	channels L,a,b,S,whiteness; Gaussian 7; Otsu hi/lo; open+close 9x9; min area 2%; score=0.4*solidity+0.4*edge/(edge+15)+0.2/(1+colour std/10); NMS IoU 0.3; thr_add 0.65 (grid 0.55-1.01)	0.447	0.540	0.489	0.830	0.470	0.46	DISCARD: Both polarities + 5 channels fixed the polarity problem and the count error (MAE 1.5 -> 0.46), but my hand-set score weights picked small corner regions over the big smooth egg.
T3	Learn the score weights (Ridge). *hand weights picked corner regions; let dev data choose*	25 features; Ridge alpha 10 on standardised features; target = IoU with GT egg; 5-fold GroupKFold on pHash groups; thr_add 0.5	0.700	0.662	0.681	0.786	0.612	0.11	DISCARD: Learning the weights (linear) helped a lot (F1 0.49 -> 0.68) but a linear score cannot express interactions such as 'large AND smooth AND high contrast'.
T4	Non-linear ranker (Random Forest). *linear score cannot express 'large AND smooth AND high contrast'*	RF 300 trees, min_samples_leaf 3, seed 0; same features/CV; thr_add 0.5	0.727	0.703	0.715	0.799	0.638	0.10	SUPERSEDED: Random forest ranker: F1 0.715. Oracle showed proposals themselves were the ceiling (only 45% of eggs had a candidate at IoU>=0.8).

Trial	What changed / why	Parameters	P	R	F1	mIoU	fgIoU	MAE	Decision
T5	Add GrabCut-refined candidates. *oracle showed proposals (not ranking) limited quality: only 45% of eggs had a candidate at IoU>=0.8*	GrabCut 4 iterations at 320 px; sure-FG = candidate eroded 21 px (11 fallback); probable-BG = dilated 29 px; top-6 candidates; second RF over raw+refined; thr_add 0.5	0.724	0.710	0.717	0.813	0.651	0.10	KEEP (FINAL): GrabCut-refined candidates added to the pool: oracle IoU>=0.8 rose 45% -> 53%, matched-egg IoU 0.80 -> 0.81, F1 unchanged. Frozen as the final method.

m-IoU: mean IoU of matched predicted/reference eggs. fg IoU: foreground (all-egg) IoU per image, averaged. P, R and F1 count a predicted egg as correct when it matches a reference egg at IoU >= 0.5 (Hungarian matching).

What the trials taught. (T0, T1) Assuming that the egg is the bright region fails because many eggs are darker than their background (cloth, trays, hands): F1 0.36-0.41. (T2) Using both polarities and five channels fixed the count (MAE 1.5 to 0.46), but my hand-set score weights picked small regions. (T3) Letting the data choose the weights (Ridge) lifted F1 from 0.49 to 0.68. (T4) A non-linear ranker gave 0.715, and an oracle check showed the candidate pool, not the ranking, was the ceiling: only 45% of eggs had a candidate at IoU >= 0.8. (T5) GrabCut-refined candidates raised that share to 53% and matched-egg IoU from 0.80 to 0.81, with F1 roughly unchanged (0.715 to 0.717). T5 was frozen as final. My honest reading: the last step improved boundary quality, not detection.

4. Task 3: evaluation against the Task 1 masks

I ran the frozen model (hash-checked) once on the 32 eval images (34 reference eggs). Predicted instances are matched to reference eggs by the Hungarian algorithm on IoU.

Metric (eval, frozen model)	Value	95% bootstrap CI	Dev out-of-fold
Precision / recall @ IoU 0.5	0.706 / 0.706		0.724 / 0.710
F1 @ IoU 0.5	0.706	0.545 - 0.849	0.717
F1 @ IoU 0.75	0.471		
Foreground IoU / Dice (per image)	0.674 / 0.772	IoU 0.585 - 0.755; Dice 0.692 - 0.840	0.651 / 0.745
Egg-count MAE / exact count	0.062 / 93.8%		0.102
False-positive regions	4		
Mean IoU of matched eggs	0.810		0.813

Reading the numbers. Counting works well (the number of eggs is right in 93.8% of images) and found eggs are outlined well (matched IoU 0.81). The weakness is *which region* is called the egg and how far it leaks: F1 drops from 0.71 to 0.47 when the IoU bar moves from 0.5 to 0.75. Eval and dev agree within the confidence interval (0.706 vs 0.717), so there is no sign of overfitting to dev. With only 32 images the interval is wide (about +/- 0.15), so differences of a few points are not meaningful.

Failure taxonomy (per reference egg, 34 total): good (IoU >= 0.75): 16; boundary error (0.5-0.75): 8; leak, where the region is far larger than the egg: 6; mislocalised: 3; merge of two eggs: 1. Plus 4 false-positive regions. Dominant failure: leak, where a white egg touches a similarly pale wall, bowl, cloth or hand and thresholding cannot separate them (see figure). Mislocalised cases are images where a bright tray or cup edge scored higher than the egg. A pure colour or edge cue has no notion of 'egg shape', which a deep model supplies.

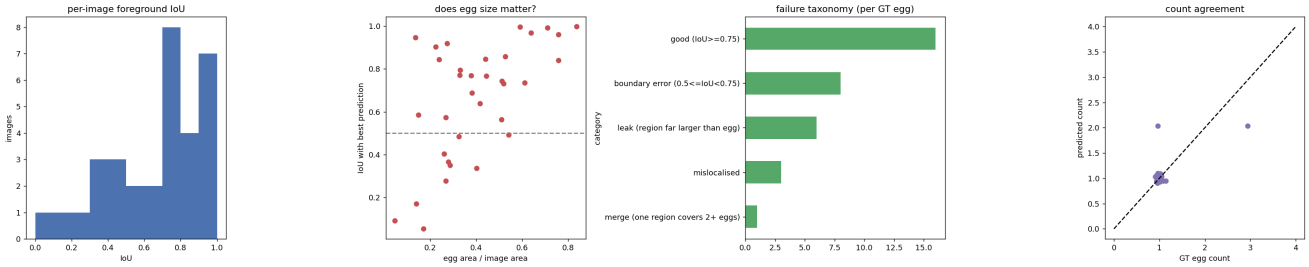

Figure 1. Eval split: per-image foreground IoU, IoU against egg size, failure taxonomy and count agreement.

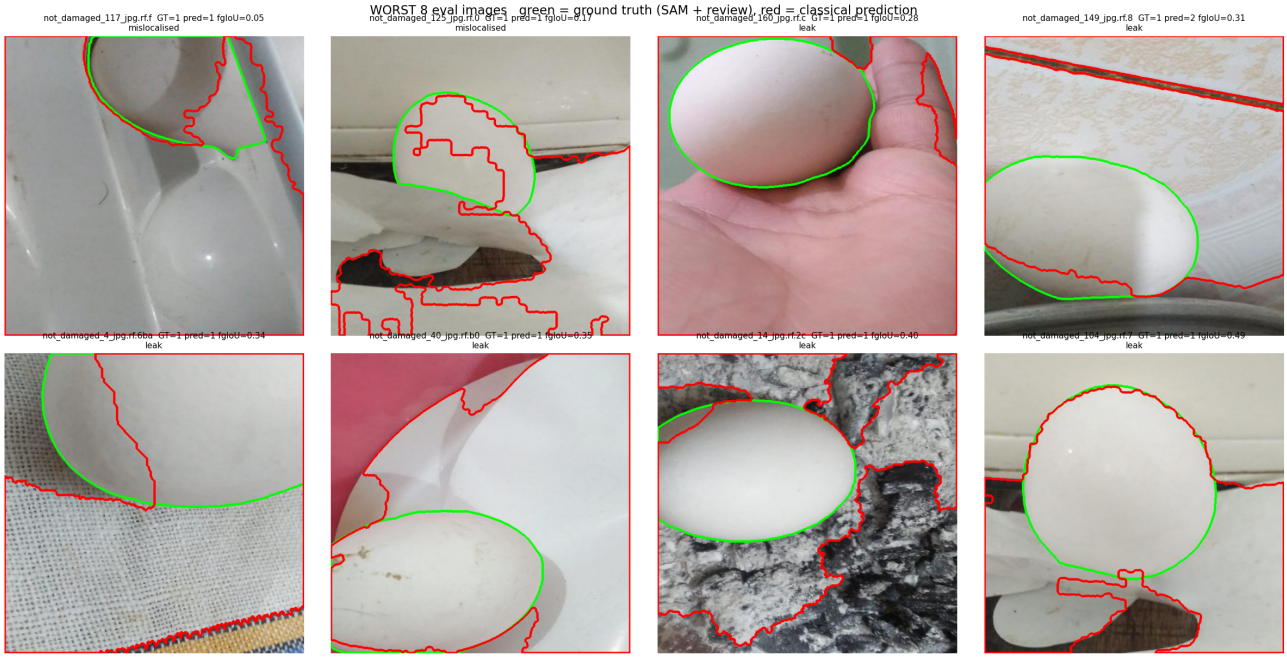

Figure 2. The eight worst eval images. Green = Task 1 mask, red = classical prediction. Mostly leaks into similarly pale surroundings, plus two mislocalised cases.

Figure 3. The four best eval images: isolated eggs with clear contrast to the background.

5. Task 4: tracking individual eggs in the video

Video: https://www.youtube.com/watch?v=Y4JyV94y2_A (1440 frames at 24 fps). **Output (public link):** https://drive.google.com/file/d/10tqBCNuAKUDZLcVYgcEYjmvncy57eFi1/view?usp=sharing. Two eggs are on screen from 9.4 s to 41.8 s; each box and mask carries a stable ID (#1, #2) and a trail.

Segmenter (video-specific). Re-running the Task 2 pipeline per frame was too slow and, with a single scene, unnecessary, so I reduced the segmenter to its essentials: work at 800 px; whiteness in LAB (L above 175, chroma below 28); morphology; distance-transform watershed to split touching eggs; shape filters on solidity, ellipse fit and minimum area (0.8% of the frame). **Check against SAM:** on 30 sampled frames (50 egg instances, 11-40 s) the masks agree with SAM (ViT-B, box prompts) with mean IoU 0.949 (median 0.962, minimum 0.819; every instance above 0.75). This tests boundary quality only, not missed eggs.

Tracker. Detections are associated to tracks with the Hungarian algorithm on a cost that combines IoU of the motion-shifted mask, centroid distance and area ratio. New objects must be seen for five frames before they become tracks. When an egg is hidden, the track coasts on a damped, capped velocity for up to 8 s with a growing gate. After the first pass, ghost tracks (seen under 1 s, or much smaller than a typical egg) are dropped and a second pass renders only the real tracks. Scene end is detected from the brightness histogram.

An error found by auditing, and the v2 fix. My first tracker (v1) looked fine in spot checks, but an identity audit (gaps over 0.5 s, jumps larger than 0.8 egg-sizes per frame, closest approach) and a frame-by-frame look at 28-34 s showed that v1 **swapped the two IDs at about 31 s**: both eggs were lost behind the hand, and the egg that reappeared was nearest to #2's last position. In version 2 I added *guided re-detection*: when a track has no clean detection, loose white blobs near its last position are matched on size, brightness, colour and solidity, so the track follows an egg while it is held in the hand and is not free to be claimed by the wrong ID. Frames produced this way are labelled '(in hand)' in the video and flagged in the tracks file. v2 is the submitted video.

Per-egg result (v2)	Egg #1	Egg #2
Track frames (9.4-41.8 s)	778	778
Detection coverage v1 -> v2	69.5% -> 81.4%	97.4% -> 92.0%
...of which in-hand re-detection frames	27	23
Longest stretch without detection (v2)	4.12 s (fully hidden)	2.08 s
Identity swaps	none seen in v2 (v1: swapped at ~31 s)	none seen in v2

Egg #2's coverage *drops* in v2 because in v1 its track had jumped to the other egg, which was cleanly detected; v2 follows the real egg #2 through the hand, so the lower number is the more correct one.

Robustness. One parameter changed at a time on the 24-38 s window: whiteness L threshold, chroma limit, solidity, minimum area and the ellipse-fit limit at 0.88 all leave the tracks unchanged (position shift 0.0-0.2 px, still 2 tracks). The exception is the ellipse-fit limit loosened to 0.72, which creates a third track and drops egg #1's detections from 312 to 223 frames; the default (0.80) is therefore close to a fragile edge, and this is a known weakness.

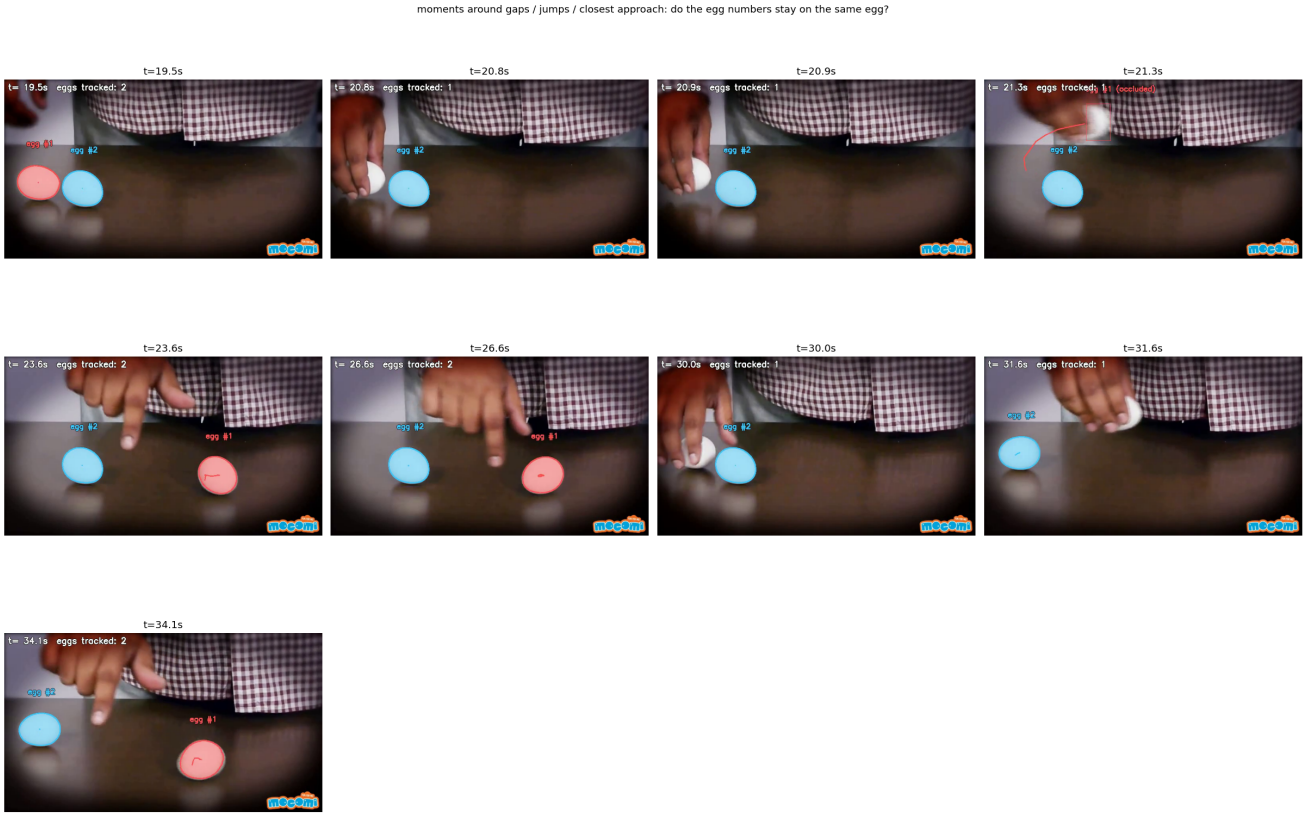

Figure 4. Identity audit (v1): detection timeline and re-links. The re-link of egg #2 at 30.4-31.2 s is where the swap happened.

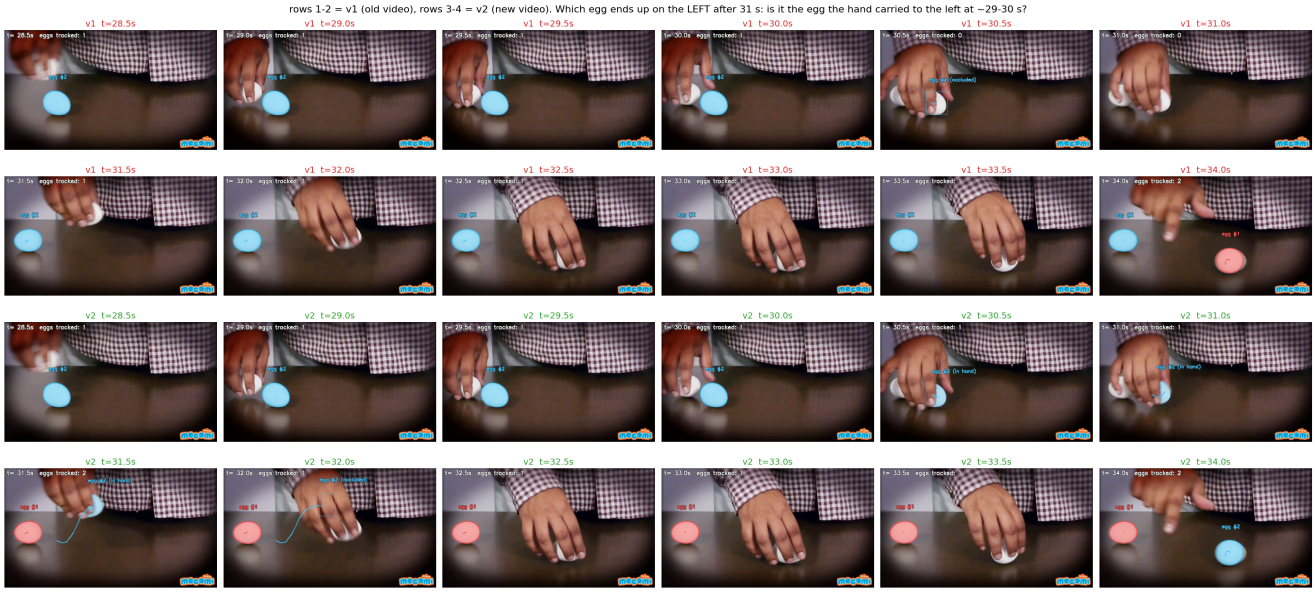

Figure 5. Side-by-side of v1 and v2 for 28.5-34 s. At 31.5 s, v2 labels the egg on the left #1 (correct, it was put down there); v1 labels it #2.

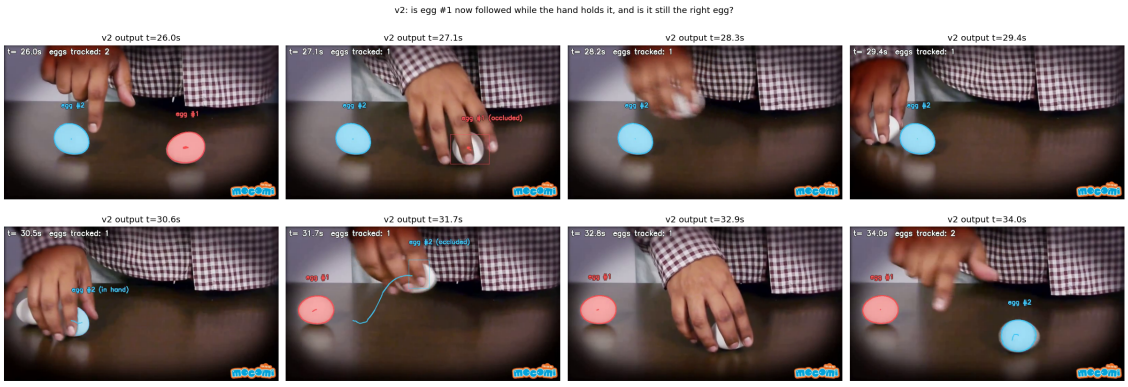

Figure 6. v2 following the eggs while held in the hand ('in hand' frames).

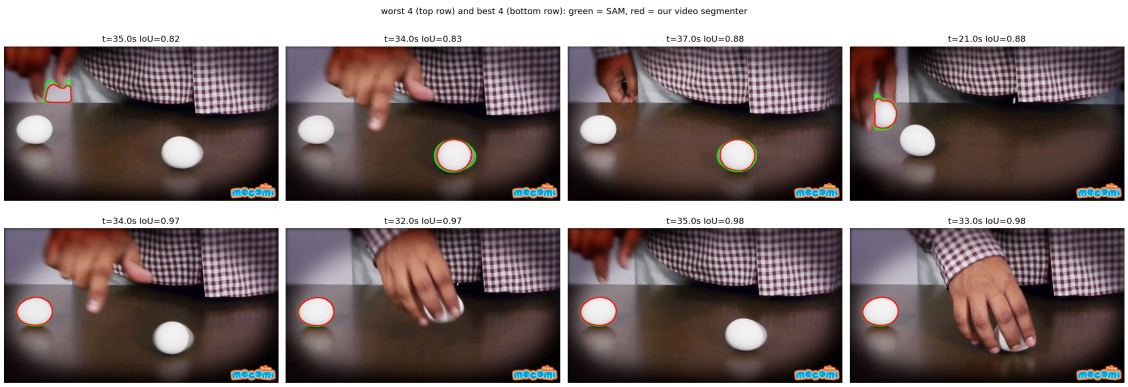

Figure 7. Video segmenter masks against SAM on sampled frames (mean IoU 0.949).

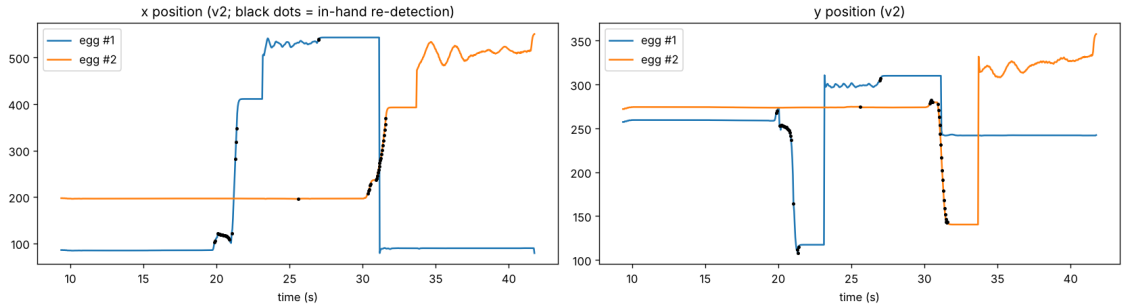

Figure 8. Egg positions over time in v2; black dots mark in-hand re-detection. Both IDs stay on continuous paths.

6. Limitations

- No ground-truth identity labels exist for the video; identity was checked by an audit and by inspecting frames, not by a numeric ID-switch metric. Absence of swaps is therefore 'none found', not proof.
- Egg #1 is not detected for up to 4.1 s while completely hidden by the hand; its position then comes from the motion model and the in-hand search.
- The video segmenter is tuned to this scene (pale eggs, one lighting). It is not a general egg detector; the Task 2 segmenter is the general one.
- Evaluation uses 32 images and 34 eggs: wide confidence intervals. The reference is SAM plus my own review, not independent annotation.
- Classical segmentation leaks into pale surroundings (6 of 34 eggs). A small learned model would likely fix most of these, but Task 2 restricts the method to classical vision.
- The dev/eval split is one random draw (seed 43); a different seed would give different eval images and slightly different scores.

7. Conclusion

I took the project from raw photos to tracked eggs in a video in four steps. Every reported number comes from a dev/eval split that I fixed before the held-out images were used.

- **Task 1:** prompting SAM with OWLv2 text detections gave usable masks for all 160 images (182 eggs). About 18% of the images (29) needed a manual fix, and each fix is recorded in the repository.
- **Task 2:** a classical propose-and-rank segmenter reached F1@0.5 = 0.717 on dev. Learning the ranking, not the thresholding, gave the biggest gain (F1 0.49 to 0.68). GrabCut refinement improved boundary quality (matched IoU 0.80 to 0.81) but not detection.
- **Task 3:** on the 32 held-out images the method scored F1@0.5 = 0.706 (95% CI 0.55-0.85), with the egg count correct in 93.8% of images. It is good at finding eggs and weaker at outlining them tightly (F1@0.75 = 0.471); the main failure is leaking into pale surroundings.
- **Task 4:** two eggs are tracked through occlusion and hand movement with stable IDs. My audit exposed an ID swap in the first tracker, and guided 'in hand' re-detection in v2 fixed it.

What I would do next: add a small learned model for the boundary, since colour and edge cues cannot tell a pale egg from a pale wall; hand-label identities in a few video segments to measure ID switches numerically; and test on more videos than this one scene.

8. AI tool disclosure

I used Claude (Anthropic) as an assistant for writing and debugging code, suggesting experiments (for example the polarity-agnostic candidates and the in-hand re-detection), checking results and drafting this report. I ran the experiments in Colab, reviewed the Task 1 masks by hand, made the keep/discard decisions, and can explain each method. Claude made one error along the way (it declared v1 free of ID swaps before the side-by-side check); my audit caught it and it is reported in section 5.

9. Reproducing the results

The code is in the public GitHub repository https://github.com/KpradeepKumar25/Egg-segmentation-and-tracking. Repository layout, setup and run commands are in its README. In short: `pip install -r requirements.txt`, set `ROBOFLOW_API_KEY` as an environment variable (never committed), then run `scripts/01_make_split.py`, `02_task1_sam_labels.py` (GPU), `03_train_classical.py`, `04_eval_task3.py` and `run_task4_video.py`. A Colab version is in `notebooks/run_all_colab.ipynb`. Frozen outputs of the submitted run are in `results/`.